

B*-Tree Index Performance

Assignment #1 — Database Systems

Case study: yarn_stash — a crocheter's yarn inventory

Database engine: PostgreSQL 18 | Table rows: 100,000

Indexes: 6 B-tree indexes including 1 expression index

Karla Budimir

June 2026

Introduction

B*-tree (Balanced-tree) indexes are the most prevalent index structure in relational databases. They allow the engine to locate rows in $O(\log n)$ time instead of performing a full $O(n)$ sequential scan, at the cost of additional disk space and write overhead. This report evaluates that trade-off experimentally using a realistic multi-column table modelling a crocheter's yarn stash, running on PostgreSQL 18.

Table Design

The `yarn_stash` table models a crocheter's personal yarn collection. Each row represents one skein of yarn and contains 14 columns of varied SQL data types, allowing index performance to be tested across different types and sizes:

Column	Type	Description
id	SERIAL PK	Auto-incrementing surrogate key
color	VARCHAR(30)	Short text — yarn colour name (30 distinct values)
purchase_date	DATE	Date the skein was purchased (2015–2024)
city_bought	VARCHAR(50)	City where yarn was purchased
country_bought	VARCHAR(50)	Country of purchase
country_origin	VARCHAR(50)	Country where yarn was manufactured
thickness	VARCHAR(20)	Yarn weight category (e.g. Medium 4)
yarn_type	VARCHAR(30)	Fibre type (Wool, Cotton, Acrylic, etc.)
weight_grams	INTEGER	Skein weight in grams (25–400g)
gauge_mm	REAL	Recommended needle size in mm (1.5–8.0)
price_eur	REAL	Purchase price in euros (1.99–29.99)
brand	VARCHAR(40)	Yarn brand name (10 distinct brands)
intended_project	VARCHAR(40)	Planned crochet project (nullable)
notes	TEXT	Free-form notes — long text, not indexed

Data Generation Script

A reusable PostgreSQL function `generate_yarn_data(n)` was implemented to insert any specified number of synthetic rows using `generate_series` and random array selection:

```
CREATE OR REPLACE FUNCTION generate_yarn_data(n INTEGER) RETURNS VOID AS $$
BEGIN
    INSERT INTO yarn_stash (color, purchase_date, city_bought, country_bought,
                           country_origin, thickness, yarn_type, weight_grams,
                           gauge_mm, price_eur, brand, intended_project, notes)
    SELECT
        (ARRAY['Cream', 'Ivory', 'Teal', 'Navy', ...])[ceil(random()*30)],
        DATE '2015-01-01' + (random()*3650)::int,
        (ARRAY['Zagreb', 'Split', 'London', 'Online', ...])[ceil(random()*10)],
        (25 + random()*375)::int,
        round((1.99 + random()*28)::numeric, 2),
        ...
    FROM generate_series(1, n);
END; $$ LANGUAGE plpgsql;

SELECT generate_yarn_data(100000);
```

Indexes Created

Six B*-tree indexes were created on columns of different types, including one expression index (upper(color)):

Index	Definition	Column type
idx_color	ON yarn_stash(color)	VARCHAR(30) — short text
idx_weight	ON yarn_stash(weight_grams)	INTEGER
idx_date	ON yarn_stash(purchase_date)	DATE
idx_brand	ON yarn_stash(brand)	VARCHAR(40) — longer text
idx_price	ON yarn_stash(price_eur)	REAL (floating point)
idx_upper_color	ON yarn_stash(upper(color))	Expression — UPPER()

Query Performance (1. & 2.)

Effect of Table Size — 1.a, 2.a

The query WHERE color = 'Teal' was executed on tables of 10k, 50k, and 100k rows, with and without idx_color. Without the index, PostgreSQL read every row (Seq Scan). With the index, it fetched only matching rows directly (Bitmap Index Scan), delivering 4–7x speedups.

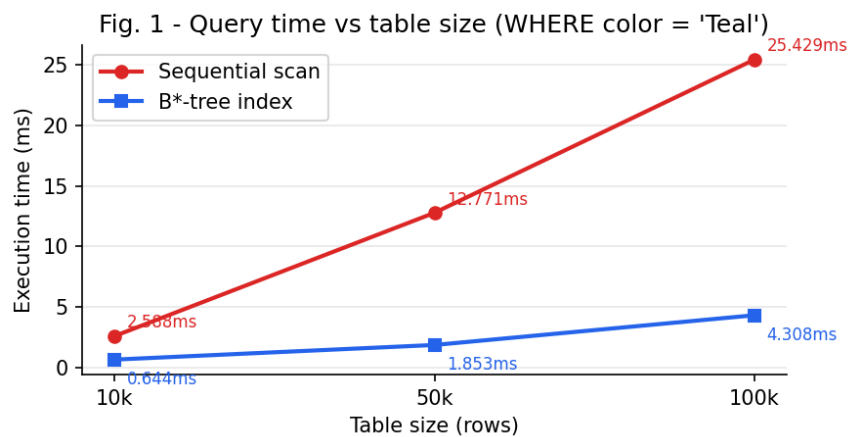


Figure 1 — Query execution time vs table size (WHERE color = 'Teal').

Sequential scan time grew linearly with table size while indexed query time stayed nearly constant. After running ANALYZE, optimizer row estimates were within 1% of actual counts at all three table sizes (e.g. estimated 3,313 vs actual 3,348 at 100k rows).

Effect of Selectivity — 1.b, 2.b

Range predicates on weight_grams (uniform 25–400g) were varied to return ~1%, ~10%, ~50%, and ~90% of rows from the 100k table.

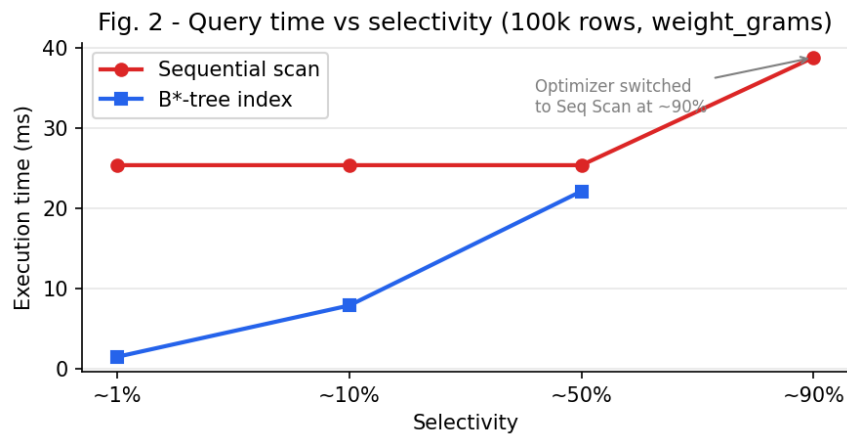


Figure 2 — Query execution time vs selectivity (100k rows, WHERE weight_grams < threshold).

At ~90% selectivity, the optimizer switched to a full sequential scan instead of using the index. At this point, so many rows matched the predicate that reading all data pages once in sequence was faster than jumping around following index pointers. For 2.b, estimates were accurate at medium selectivity (~1% error at 10–50%) but showed ~31% error at very low selectivity (<1%), due to histogram bucket granularity.

Effect of Column Type & Expression — 1.c, 2.c

Six queries with similar selectivity (~3–11%) were run against all indexed column types using EXPLAIN ANALYZE:

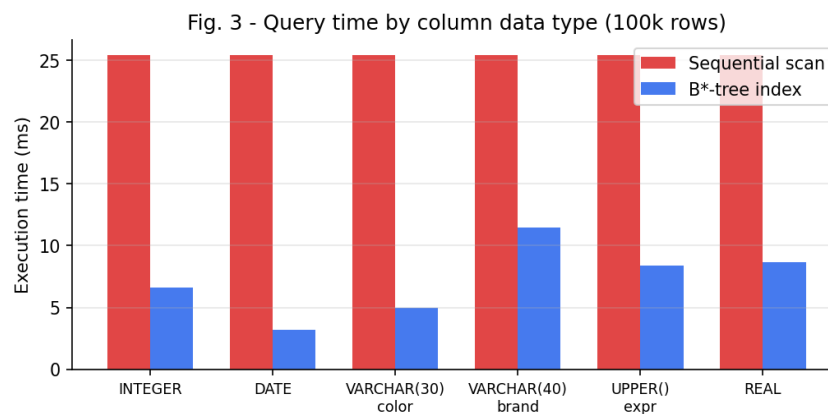


Figure 3 — Query time by indexed column/expression type (100k rows).

Column	Type	Seq scan	Index time	Speedup	Est. rows	Actual rows
weight_grams	INTEGER	25.4 ms	6.6 ms	~4x	5,649	5,561
purchase_date	DATE	25.4 ms	3.2 ms	~8x	2,574	2,562
color	VARCHAR(30)	25.4 ms	5.0 ms	~5x	3,313	3,348
brand	VARCHAR(40)	25.4 ms	11.5 ms	~2x	9,557	9,798
upper(color)	Expression	25.4 ms	2.8 ms	~9x	3,313	3,348
price_eur	REAL	25.4 ms	8.7 ms	~3x	10,981	11,089

DATE indexes performed best (3.2 ms). The expression index on upper(color) correctly intercepted WHERE upper(color) = 'TEAL' and delivered the best speedup (~9x) after ANALYZE improved its estimate from 500 to 3,313 rows — demonstrating that ANALYZE is critical for expression index accuracy. All optimizer estimates were within 2.5% of actual after ANALYZE (2.c).

DML Performance Impact (3.)

DML stands for Data Manipulation Language which refers to SQL commands that modify the data inside tables. DML operations, INSERT, UPDATE and DELETE, were timed with and without all 6 indexes on the 100k-row table using \timing on in psql:

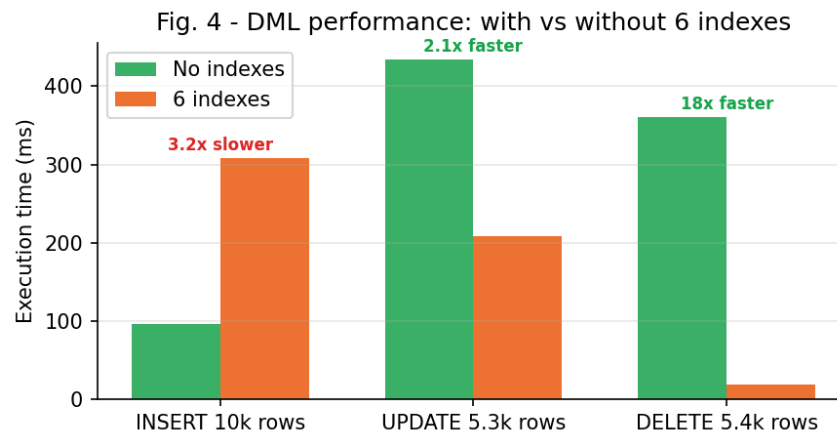


Figure 4 — DML execution time with vs without 6 B*-tree indexes.

Operation	Without indexes	With 6 indexes	Impact
INSERT 10,000 rows	96 ms	308 ms	3.2x slower
UPDATE 5,342 rows (color=Teal)	434 ms	209 ms	2.1x faster
DELETE 5,440 rows (color=Cream)	360 ms	19.5 ms	18x faster

INSERT operations were 3.2 times slower with indexes because each new row had to be added to all six index structures. In contrast, UPDATE and DELETE operations were faster because PostgreSQL used the index on the color column to locate matching rows directly, avoiding a full table scan. This demonstrates that while indexes increase the cost of inserting data, they can significantly improve the performance of updates and deletes when the WHERE clause uses an indexed column.

Index Disk Space (4.)

Disk Space vs Table Size — 4.a

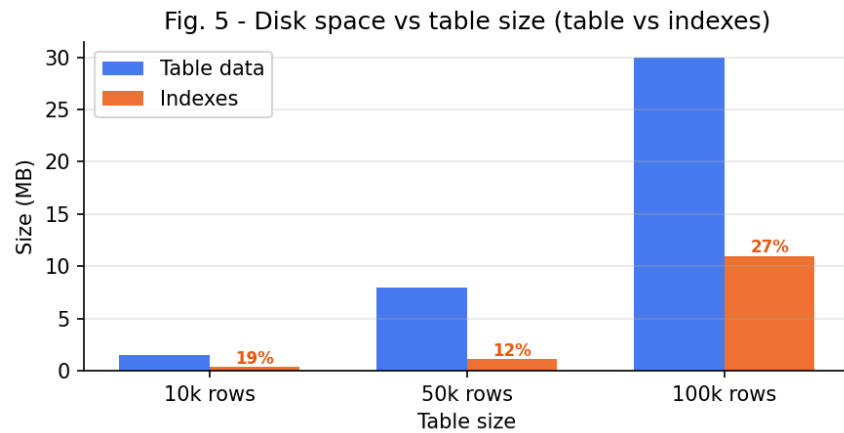


Figure 5 — Table vs index disk space at different table sizes.

Table size	Table data	Index size	Index % of total
10,000 rows	1,544 kB	352 kB	19%
50,000 rows	8,192 kB	1,168 kB	12%
100,000 rows	30 MB	11 MB	27%

Index size scales linearly with row count. With the full set of 6 indexes at 100k rows, total index storage reaches 37% of table size.

Disk Space vs Column Type — 4.b

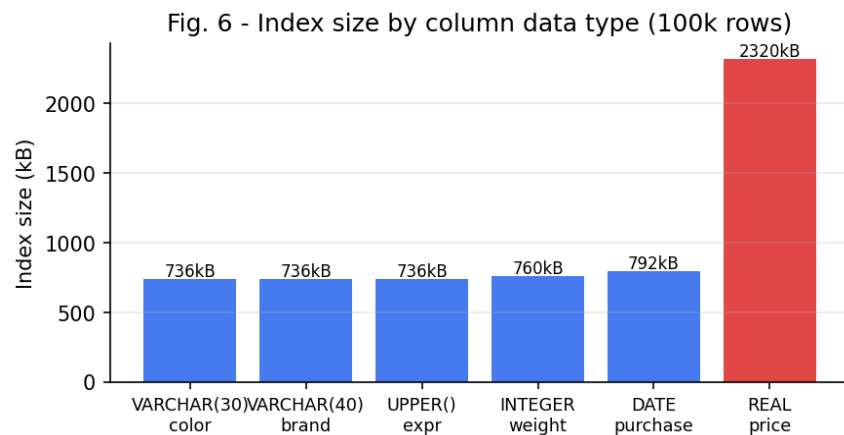


Figure 6 — Individual index size by column data type (100k rows).

Index	Column type	Size	Notes
idx_color	VARCHAR(30)	736 kB	30 distinct values, low cardinality
idx_brand	VARCHAR(40)	736 kB	10 distinct values, low cardinality
idx_upper_color	Expression UPPER()	736 kB	Same cardinality as color
idx_weight	INTEGER	760 kB	Fixed 4-byte values
idx_date	DATE	792 kB	Fixed 4-byte values, wide date range
idx_price	REAL	2,320 kB	Nearly unique per row — high cardinality

Most indexes were similar in size (736–792 kB). The exception was idx_price at 2,320 kB — nearly 3x larger — because floating-point prices are nearly unique per row (high cardinality), producing a much denser B*-tree with minimal key sharing between leaf nodes.

5. Index Build Time

Build Time vs Table Size — 5.a

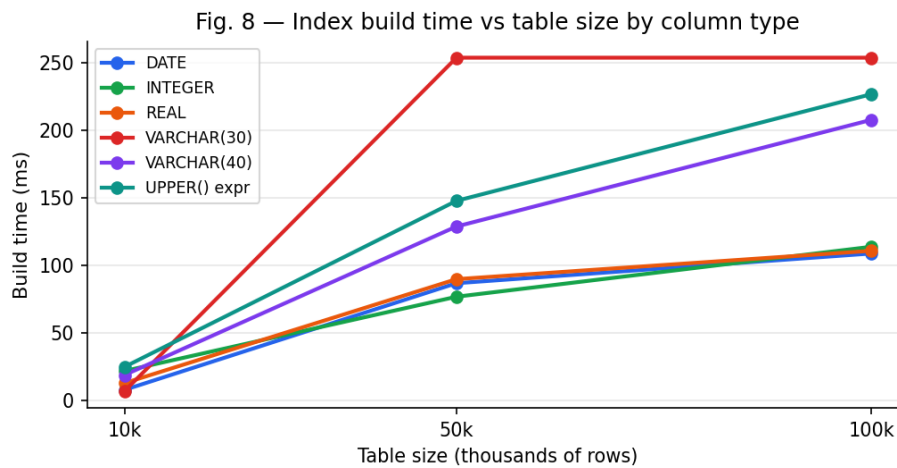


Figure 7 — Index build time vs table size for each column type.

Column type	10k rows	50k rows	100k rows
DATE (purchase_date)	8 ms	87 ms	109 ms
INTEGER (weight_grams)	22 ms	77 ms	114 ms
REAL (price_eur)	13 ms	90 ms	111 ms
VARCHAR(30) color	7 ms	254 ms	254 ms
VARCHAR(40) brand	19 ms	129 ms	208 ms
UPPER(color) expression	25 ms	148 ms	227 ms

Build time scales linearly with row count across all column types, consistent with the $O(n \log n)$ sort cost of bulk index construction. At 100k rows the spread between fastest (DATE, 109 ms) and slowest (VARCHAR(30), 254 ms) is ~2.3x.

Build Time vs Column Type — 5.b

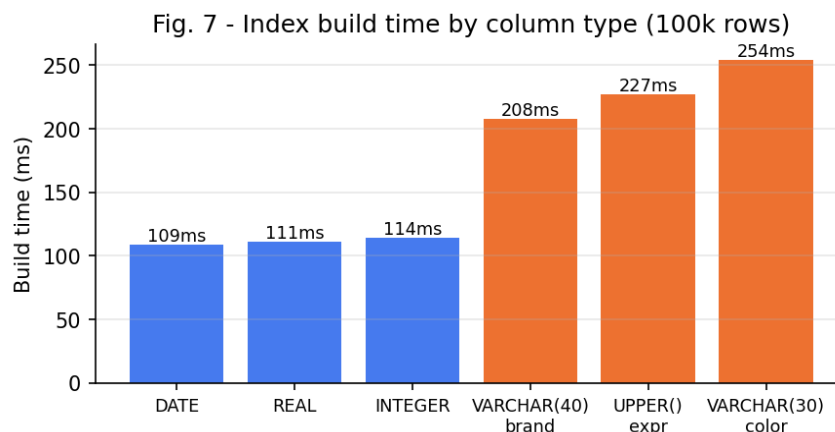


Figure 8 — Index build time by column type (100k rows).

INTEGER and DATE indexes build fastest (109–114 ms) due to fixed-size and low comparison cost. VARCHAR indexes are slower (208–254 ms) due to string comparison cost. The expression index UPPER() was among the slowest (227 ms) since PostgreSQL must evaluate the expression for every row during the sort phase.

Conclusion

B*-tree indexes deliver dramatic query speedups when the WHERE condition matches only a small fraction of rows, and scale logarithmically with table size. The trade-offs — disk overhead (~37%) and INSERT penalties (3.2x) — are predictable and manageable. Index build time scales linearly with row count, with numeric and date indexes building roughly twice as fast as VARCHAR or expression indexes. UPDATE and DELETE operations can benefit from indexes when the WHERE column is indexed. PostgreSQL's optimizer reliably selects the correct access path after ANALYZE, with near-perfect row estimates across all column types and table sizes.